

UNIT-3

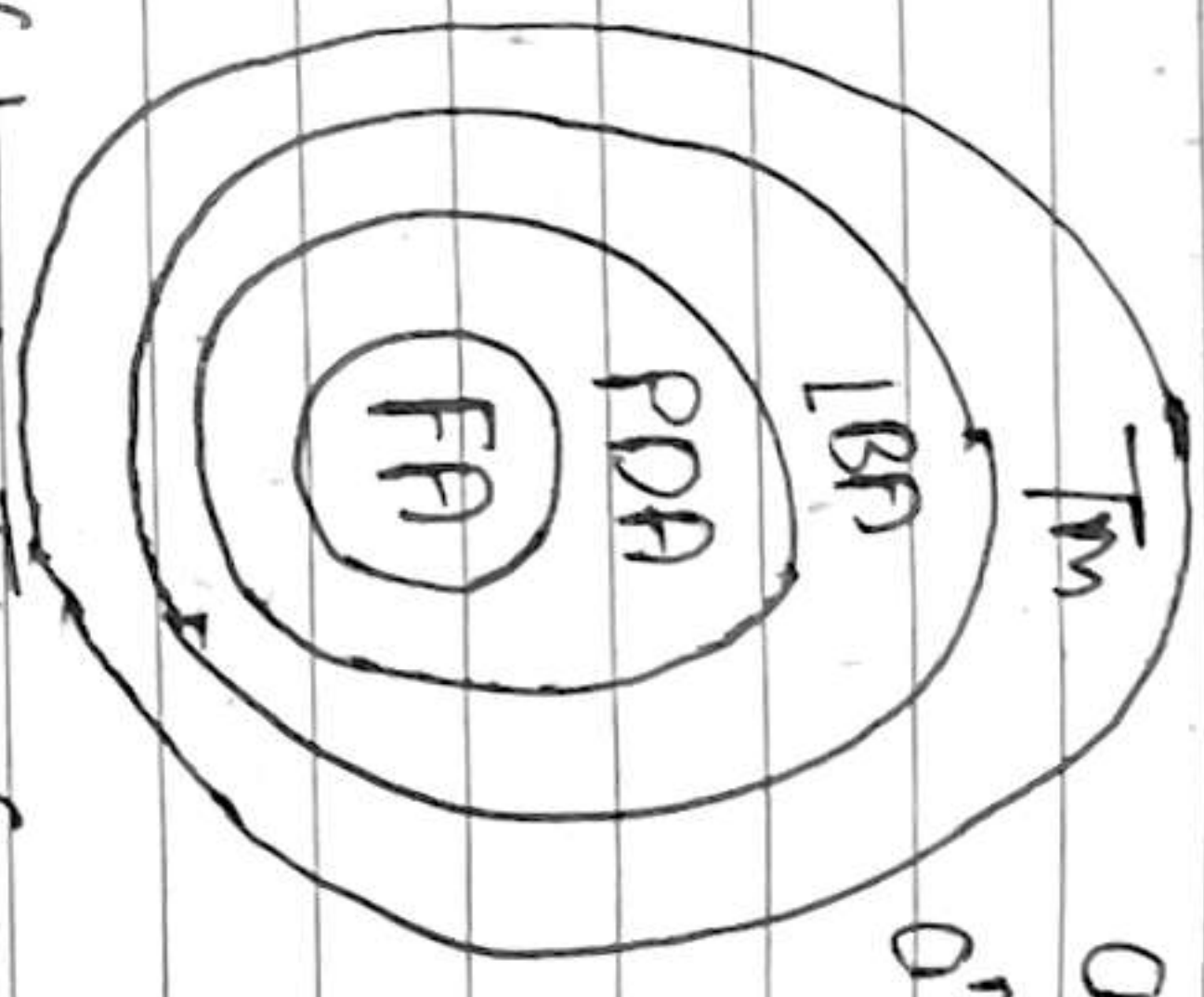
Introduction to Turing Machines

only pairs or

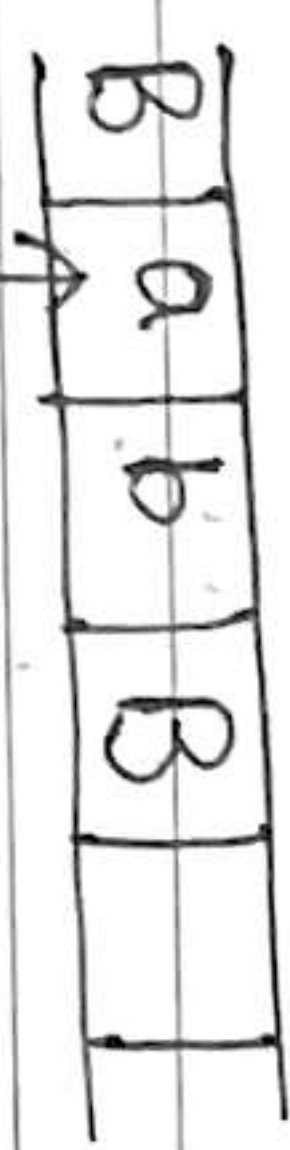
Turing Machine

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

$$a^n b^n \Rightarrow PDA$$



- Q = states
- Σ = input symbols
- Γ = symbols allowed in tape ($\Sigma + B$)
- δ = ~~partial~~ transition function
- q_0 = initial state
- B = Blank
- F = final states



$$T(n) = O(n \log n) = \text{Time Complexity}$$

$$S(n) = O(n) = \text{Space Complexity}$$

1) Read & write allowed
2) left, right allowed

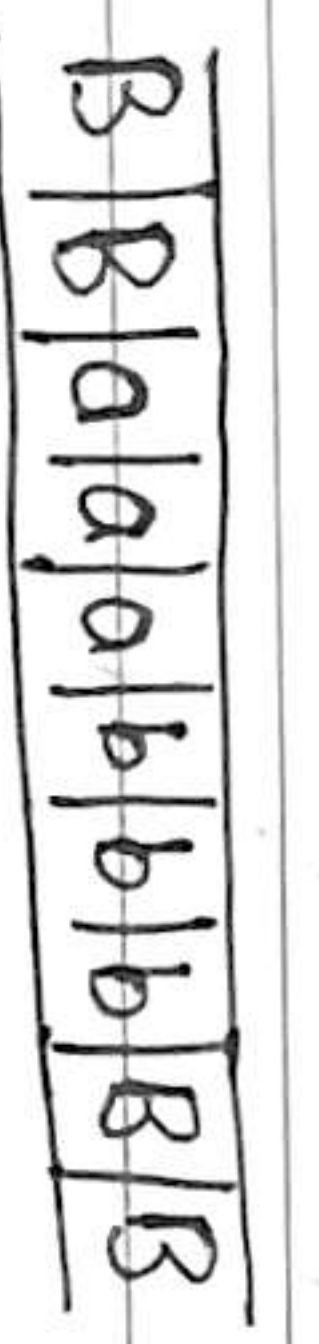
$$\delta = Q \times \Gamma \rightarrow Q \times \Gamma \times (L, R)$$

$$(q_0, a) \rightarrow (q_1, x, R)$$

current state q_0
input symbol a
replaced by x

Finite Control

Q. Design Turing Machine for $\{a^n b^n | n \geq 1\}$



R/w (y, y, R)

FC



B/x/a/y/b/B

B/x/a/b/b/B

B/x/x/y/b/B

B/x/x/y/y/B

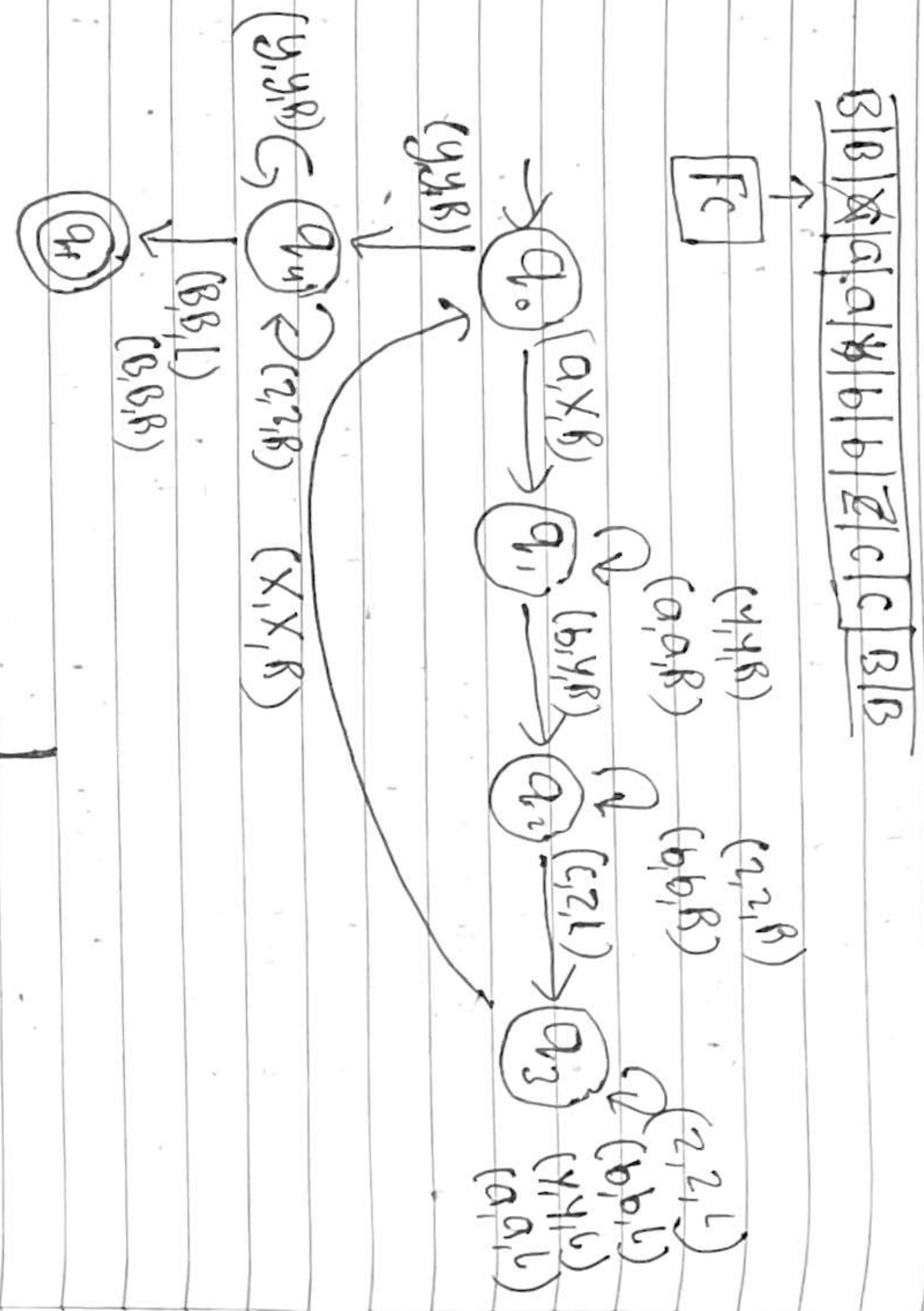
Instance Description (ID) for the string "aacabb"

BaacbbbB $\rightarrow$ BxaaabbbbB
 BxaaaabbbB $\rightarrow$ BxaaaabbbB
 BxaaaabbbB $\rightarrow$ BxaaaabbbB

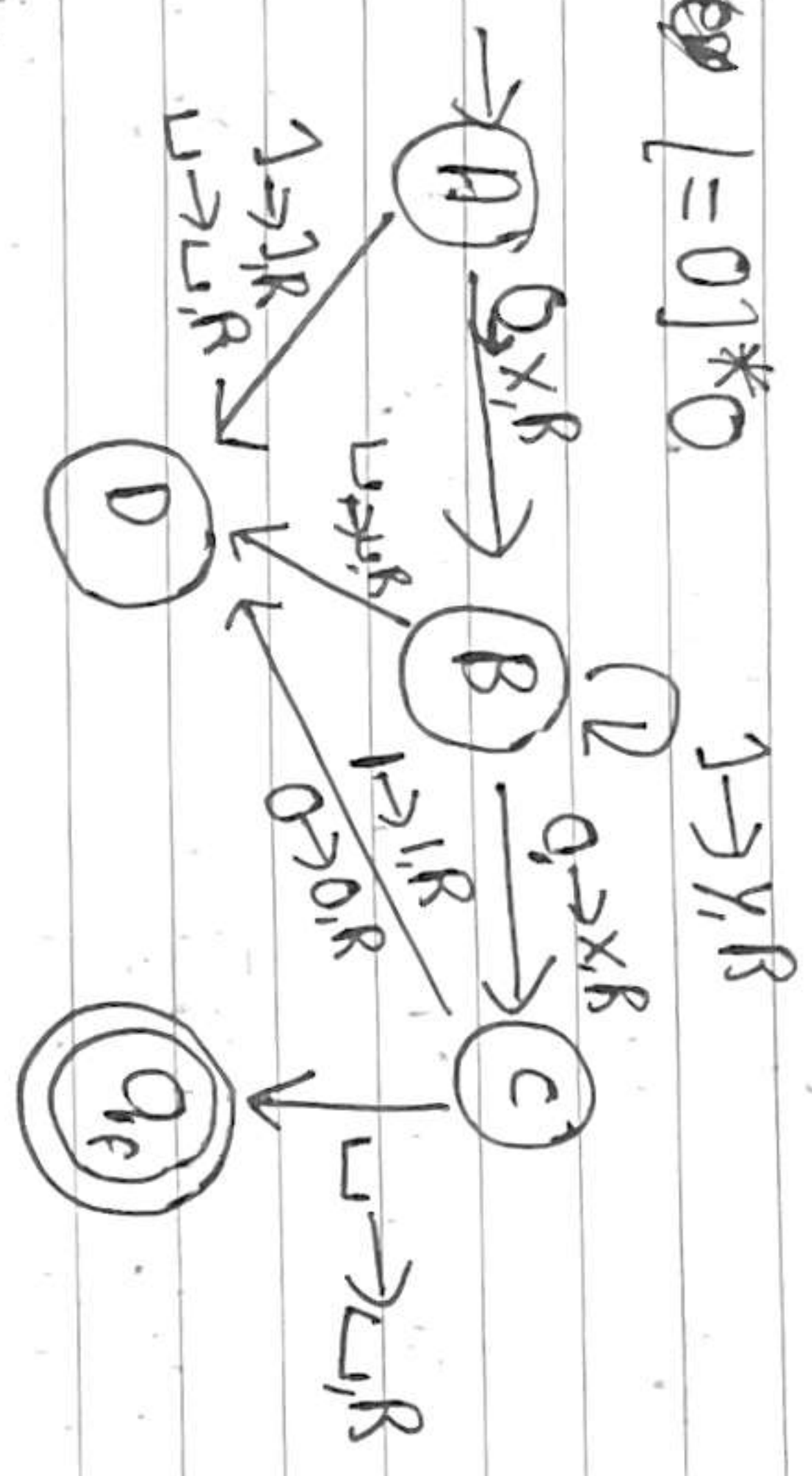
BxxxxyybbB $\rightarrow$ BxxxxyyyyB (Halt in accepting state)

Q4

Q. $\{a^n b^n c^n | n \geq 1\}$



Q. $\{a^n | n \geq 0\}$



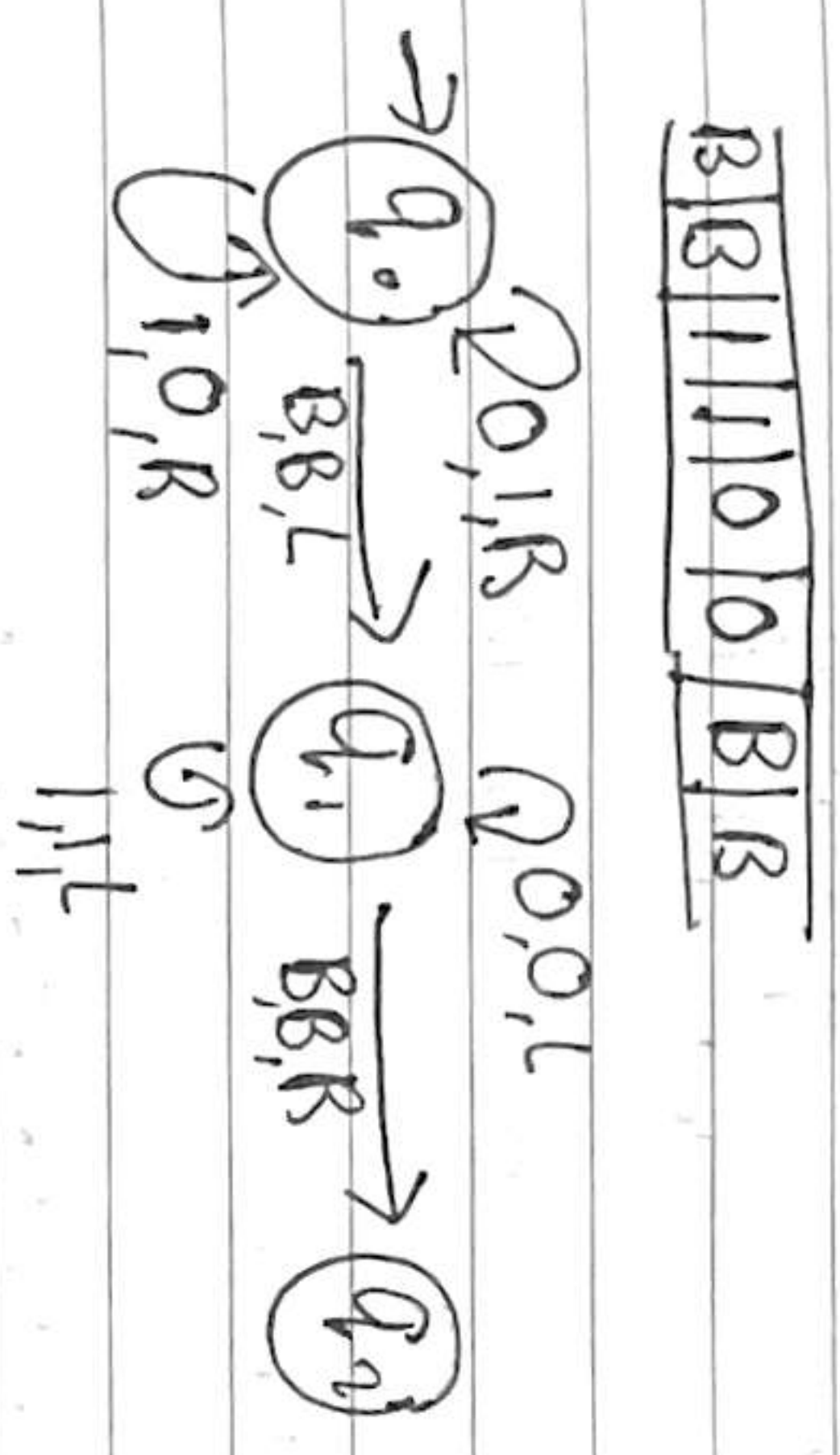
Tape

REC RE

Properties of Recursive & Recursively enumerable (Type-0) languages

Recursive Language	Recursively Enumerable Language
(i) It is recursive if there is halting/total TM	(i) It is RE if there is TM
(ii) 2 states: Halt & accept Halt & reject	(ii) 3 states: Halt & accept Halt & reject Never Halt
(iii) Closed under all except homomorphism & substitution	(iii) Closed under all except — complement (converts input)

Q. Turing Machine for 1's complement (Tosmduend)



If a problem can be solved by any algorithm, it can be solved by a TM

CHURCH - TURING Thesis -

What does Computable mean?

Several variations of Turing Machine -

- (i) One tape on many
- (ii) Infinite on both ends
- (iii) Alphabets only {0,1} on more
- (iv) Can the head also stay in the same place
- (v) Allow non determinism

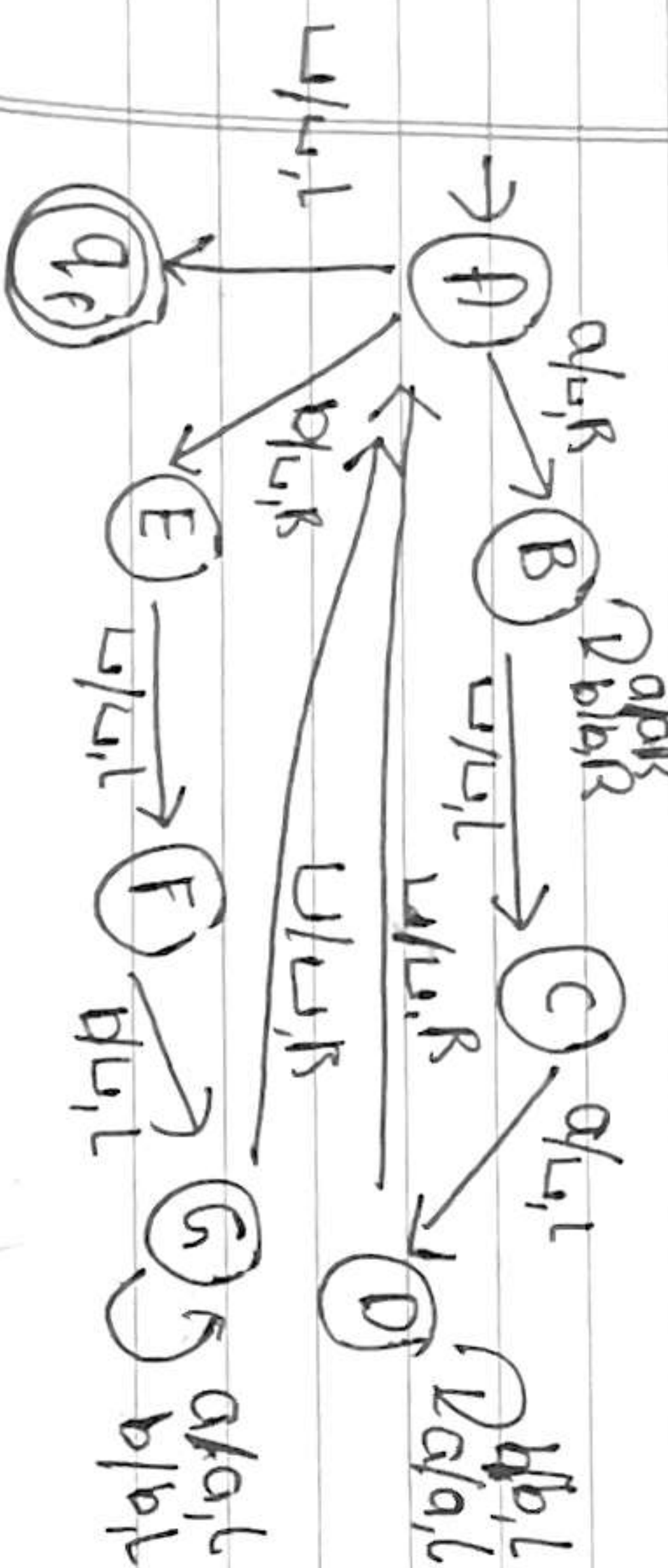
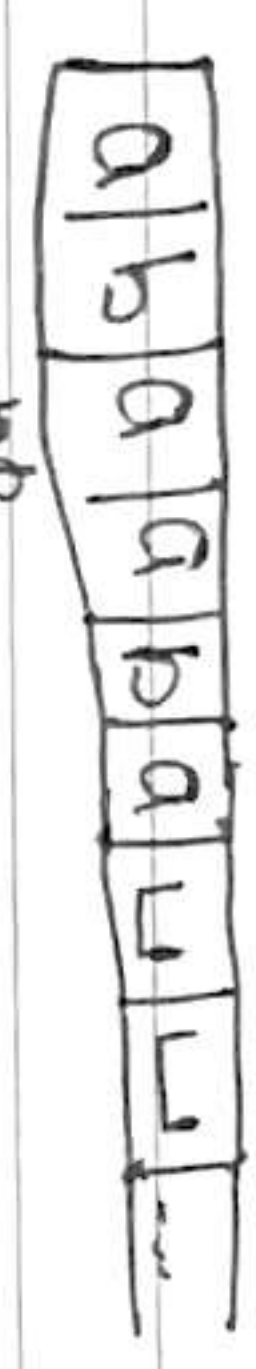
=> All variations are equivalent in computing capability

=> Turing Machine & Lambda Calculus are also equivalent in power

=> Algorithmically computable means computable by a TURING MACHINE

Q. Turing Machine for Even Palindrome -

eg. abbaab



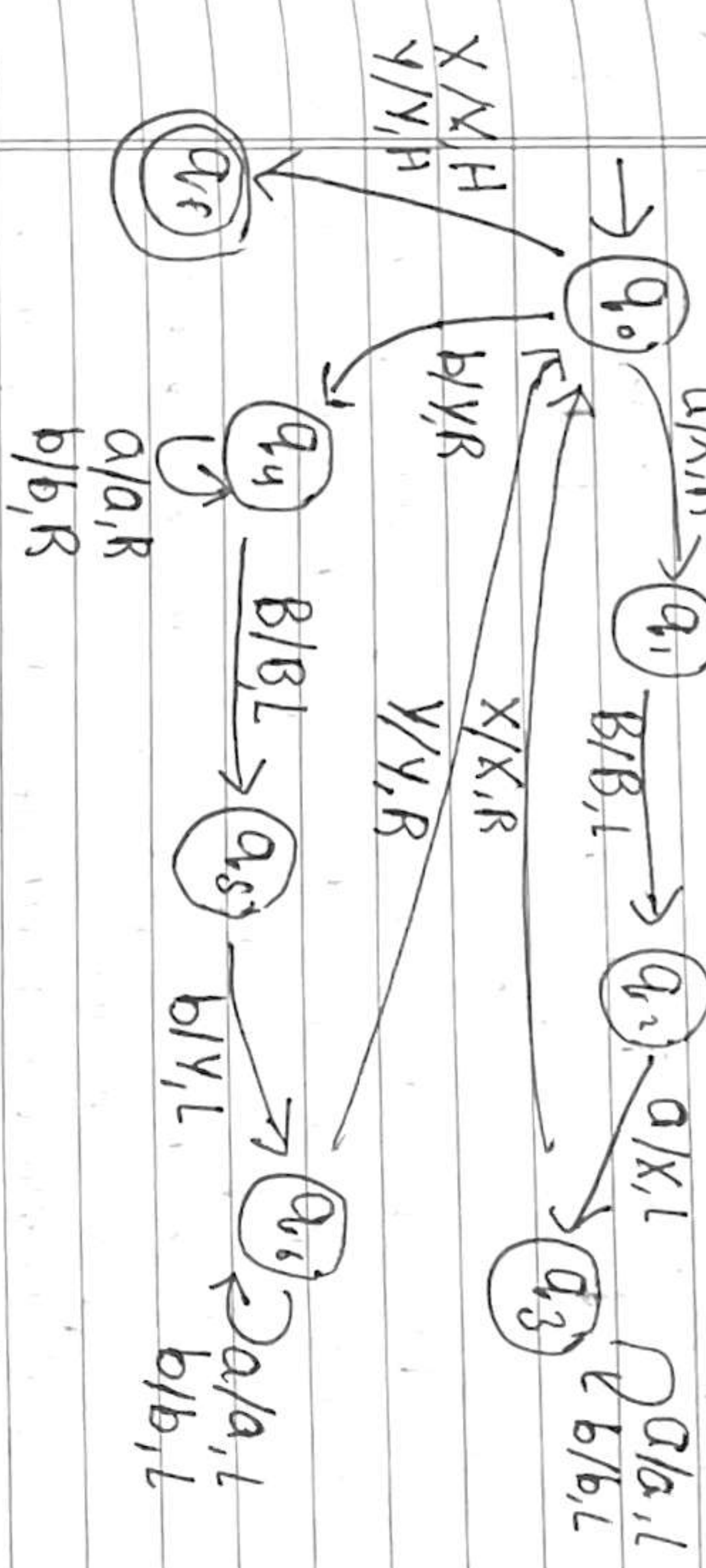
Importance -

- 1) Defines computability
- 2) Standard for algorithms
- 3) Basis for computability
- 4) Foundation of C.S.

Comparison with previous automata -

Machine	Stack Data Structure	Deterministic
Finite Automaton	NA	Yes
Pushdown Automaton	Last In First Out (LIFO)	No
Turing Machine	Infinite tape	Yes

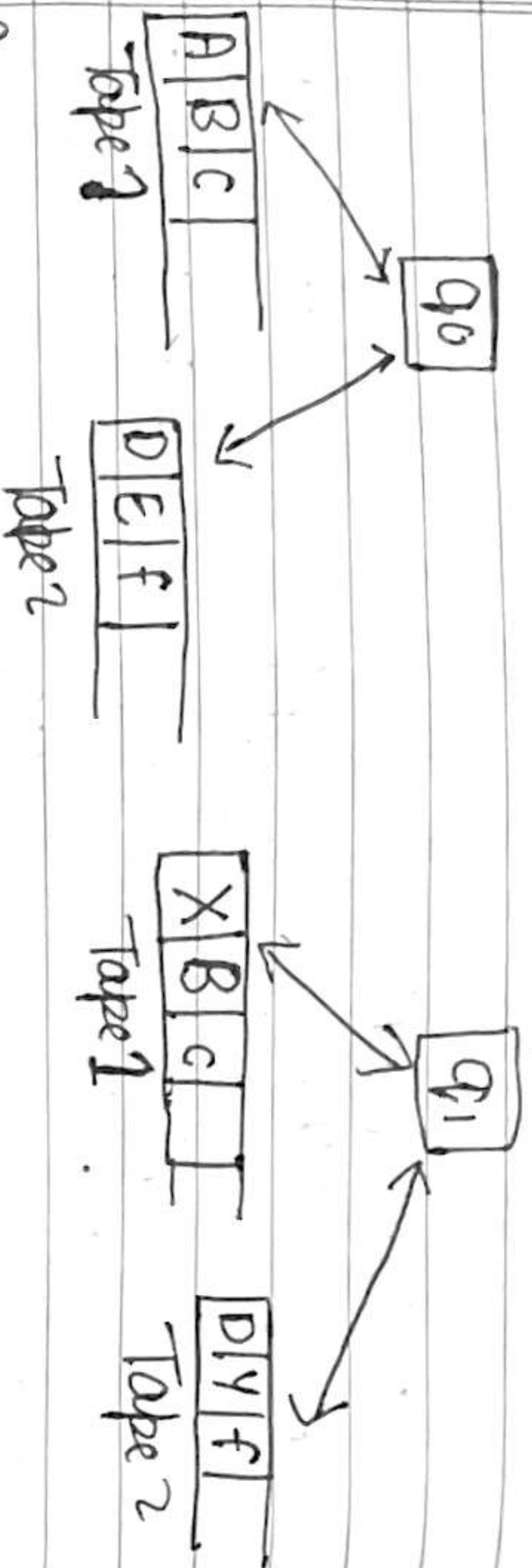
Q. Turing Machine for ww^R AND $t = reverse$



$(q_0, abaaabab) \rightarrow (q_1, abaaabab)$
 $\rightarrow (q_1, xbaaaabab) \rightarrow (q_2, xbaaaabab) \rightarrow (q_3, xbaaaabab) \rightarrow (q_4, xyaaaaabx) \rightarrow (q_5, xyaaaaabx) \rightarrow (q_6, xyaaaaqx) \rightarrow \dots$
 $xyxxxxxyx$

Multi tape Turing Machine -

- 1) An extension of the standard Turing machine where multiple tapes are available for input, output and computation.
- 2) Each tape has its own read-write head & the machine is transition function is based on the current state and the symbols read by each head.

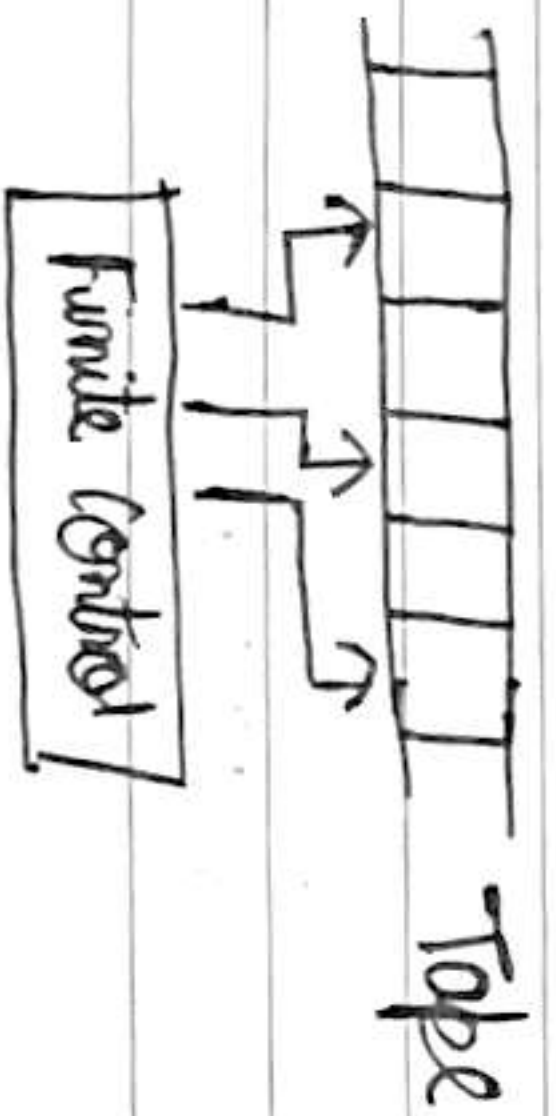


eg - Checking Palindrome using 2 tape Turing Machine

- (i) A 2 tape Turing machine copies the input binary string from tape 1 to tape 2
- (ii) It then compares symbols from both tapes in opposite directions
- (iii) If all symbols match the string is a Palindrome
- (iv) This approach is faster & more efficient than using a single tape machine

Multi head Turing Machine -

- 1) A single tape is used, but it has multiple read/write heads.
- 2) These heads can independently read & write symbols, enabling the machine to perform complex tasks more efficiently.



eg - Palindrome

- (i) 2 heads start at both ends of the string & move towards the center, comparing symbols
- (ii) If both symbols match the process continues, otherwise it stops (not a palindrome)
- (iii) If multiple heads write simultaneously, a priority rule decides which action happens
- (iv) A 'hanging' condition occurs if a head moves left beyond the first cell and must be handled.
- (v) A multi head machine can still be converted to a single head Turing machine, though with more complexity

2 way Infinite Tape Turing Machine -

- 1) Can be particularly useful in computations where data may need to be processed symmetrically around a central point
- 2) Can be simulated by a standard Turing machine by making the left boundary & preventing the head from moving beyond it

K-dimensional Turing Machine -

- 1) Extends concept of the tape to multiple dimensions
- 2) 2D Turing machine the read-write head can move not only left and right but also up and down
- 3) Used in image processing or simulations of physical phenomena

Non deterministic Turing Machine -

- 1) Can move to multiple possible states from one config
- 2) Each NDTM has an equivalent deterministic TM, which simulates all paths & accepts if any path reaches an accepting state

Enumerated Turing Machine -

- 1) Generates all possible strings of a language
- 2) Uses a work tape & an output tape to write & print strings
- 3) After printing one string, it resets to create the next
- 4) Useful for listing all valid strings or solutions in a problem

Equivalence with Single Tape Turing Machine -

- 1) Extends the single tape Turing machine by using multiple tapes, each with its own read/write head
- 2) Though it appears more powerful, it can be simulated by a single tape TM, so both have equal computational power
- 3) Difference lies in efficiency - multitape TMs perform computations faster
- 4) A Turing recognizable (recursively enumerable) language is one that a TM (single or multitape) accepts if it belongs to the language and rejects or loops otherwise
- 5) Preserves both models recognize the same class of languages

Turing Recognizable vs Turing Decidable (Recursively Enumerable) (Recursive)

Aspect	Turing Recognizable	Turing Decidable
1) Definition	It there exists a TM that accepts all strings in the language but may loop forever for others	It there exists a TM that accepts strings in the language & rejects others always halting
2) Machine Behavior	TM may not halt for some inputs not in the language	TM always halts for every input

3) Acceptance TM accepts strings in the language but may not reject in conditions

TM accepts or rejects every input correctly

4) Example When TM encounters a string not in the language, it may loop indefinitely

When TM encounters a string not in the language, it halts & rejects it

5) Strength Weaker condition than Turing Decidable

Stronger condition than Turing Recognizable

Single Tape TM vs Multitape TM

Feature Definition
Single Tape TM
TM with one tape & one head

Multitape TM
TM with multiple tapes and heads

1) Computational Power
Equal to multitape TM

Equal to single tape TM

2) Efficiency
Less efficient
More complex

More efficient
Easier due to separate tapes

3) Design
Can simulate multitape TM with slowdown

Can be simulated by single tape TM

6) Recognizable language
Turing recognizable language

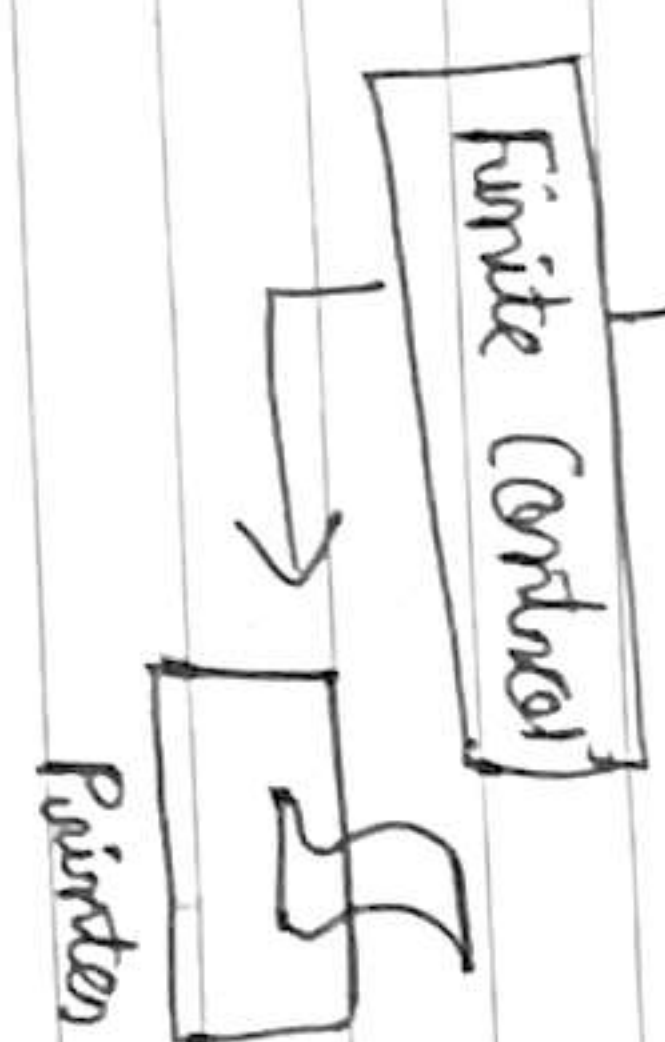
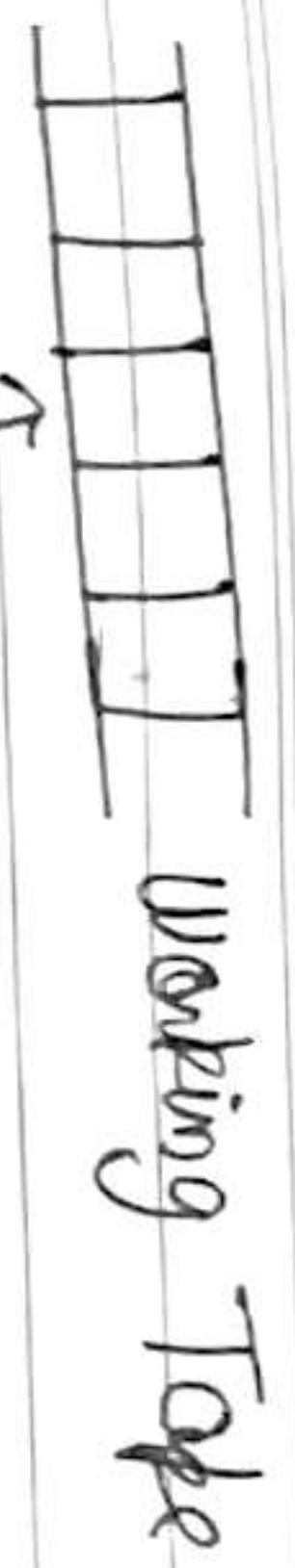
Also recognizes Turing recognizable languages

Deterministic Turing Machine (DTM) vs Non Deterministic Turing Machine (NDTM)

Feature	DTM	NDTM
1) Transition	One specific action for each (state, symbol) pair	Multiple possible actions for a single (state, symbol) pair
2) Behaviour	Follows a single computation path	Can explore multiple paths simultaneously (conceptually)
3) Execution	Step by step & predictable	Guesses the correct path non deterministically
4) Acceptance	Accepts input if it reaches an accepting state	Accepts input if any path reaches an accepting state
5) Efficiency	Longer for complex problems	Can solve problems faster
6) Implementation	Practically implementable	Theoretical model only

Enumerates Turing Machine -

- 1) A variation of the Turing machine used to generate or list strings of a language.
- 2) It outputs strings belonging to the language.
- 3) Works by printing strings one at a time, possibly in any order.
- 4) Has an infinite tape & finite state control like a normal TM.
- 5) Key difference is that it includes a printer (output tape).
- 6) Printer allows it to produce strings sequentially.
- 7) Useful for listing all valid strings/solutions of a problem.



Key Characteristics

- 1) Tape = Initially empty; no input string is provided (unlike standard TM)
 - 2) Finite State = Works like a Turing machine's control, guiding operations based on current state and symbols.
 - 3) Printer = Unique feature that outputs strings belonging to the language being enumerated.
 - 4) Operations = Starts with an empty tape, generates strings, & prints them - listing all strings of the language.
 - 5) Halting & = May stop after producing some strings or run indefinitely for infinite language, printing endlessly.
- ## # Enumerability & Enumeration Definition -
- A language is enumerable if an enumerator can generate all its valid strings.
- Enumerable languages are the same as Turing recognizable languages.
- If a TM accepts a language, an enumerator can list its strings, & vice versa.

- 4) A Turing machine decides by accepting or rejecting strings
- 5) An enumerator only prints the valid strings of the language
- 6) Set of printed strings represents the language it defines
- 7) An enumerator can print duplicate strings; repetition is allowed & valid
- 8) Order of printing doesn't matter - strings can appear in any sequence
- 9) Only requirement is that every string in the language must be printed at least once
- 10) Enumerators have same computational power as Turing Machine

Enumeration → Turing Machine

- 1) TM sums the enumeration, generates strings, & compares them with the input
- 2) If a match is found, TM accepts the input
- 3) For infinite languages, it keeps comparing indefinitely until a match appears

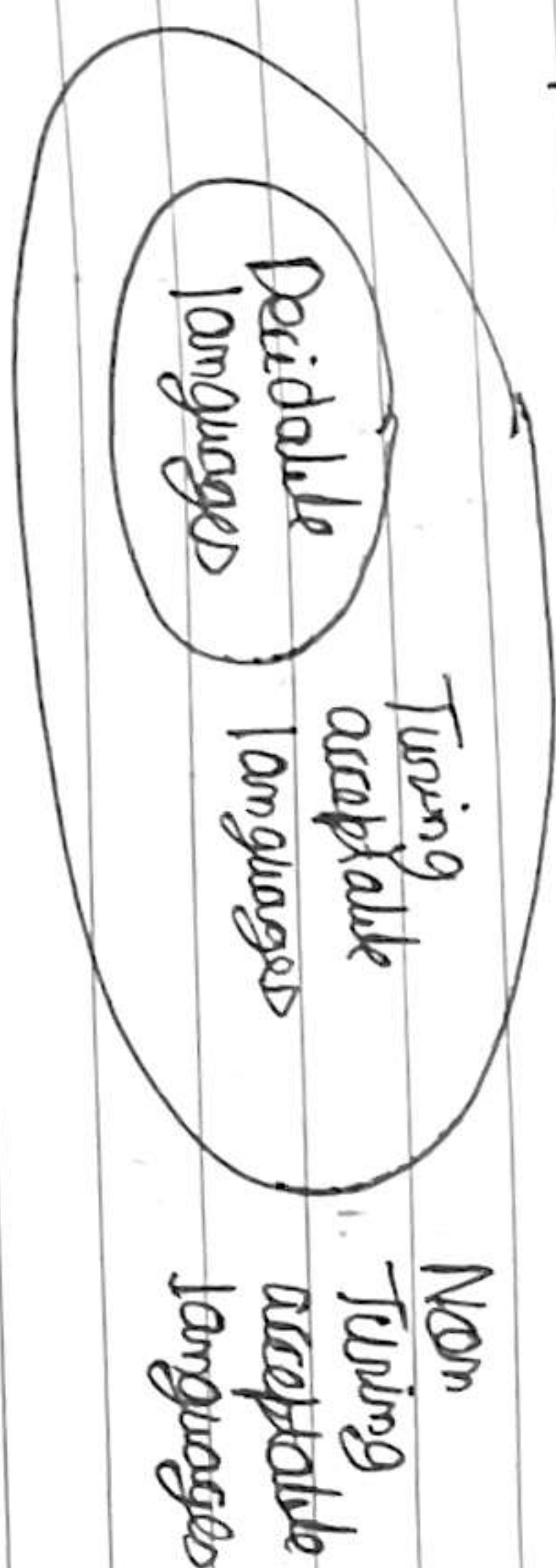
Turing Machine → Enumerator

- 1) It generates all possible strings, sums the TM on each, and prints accepted strings
- 2) To handle looping, it uses parallel computation - testing multiple strings in steps to ensure none are missed
- 3) Even if the TM ~~also~~ loops for some ~~the~~ inputs, the enumeration still prints valid strings belonging to the language

Church Turing Thesis =

Decidability -

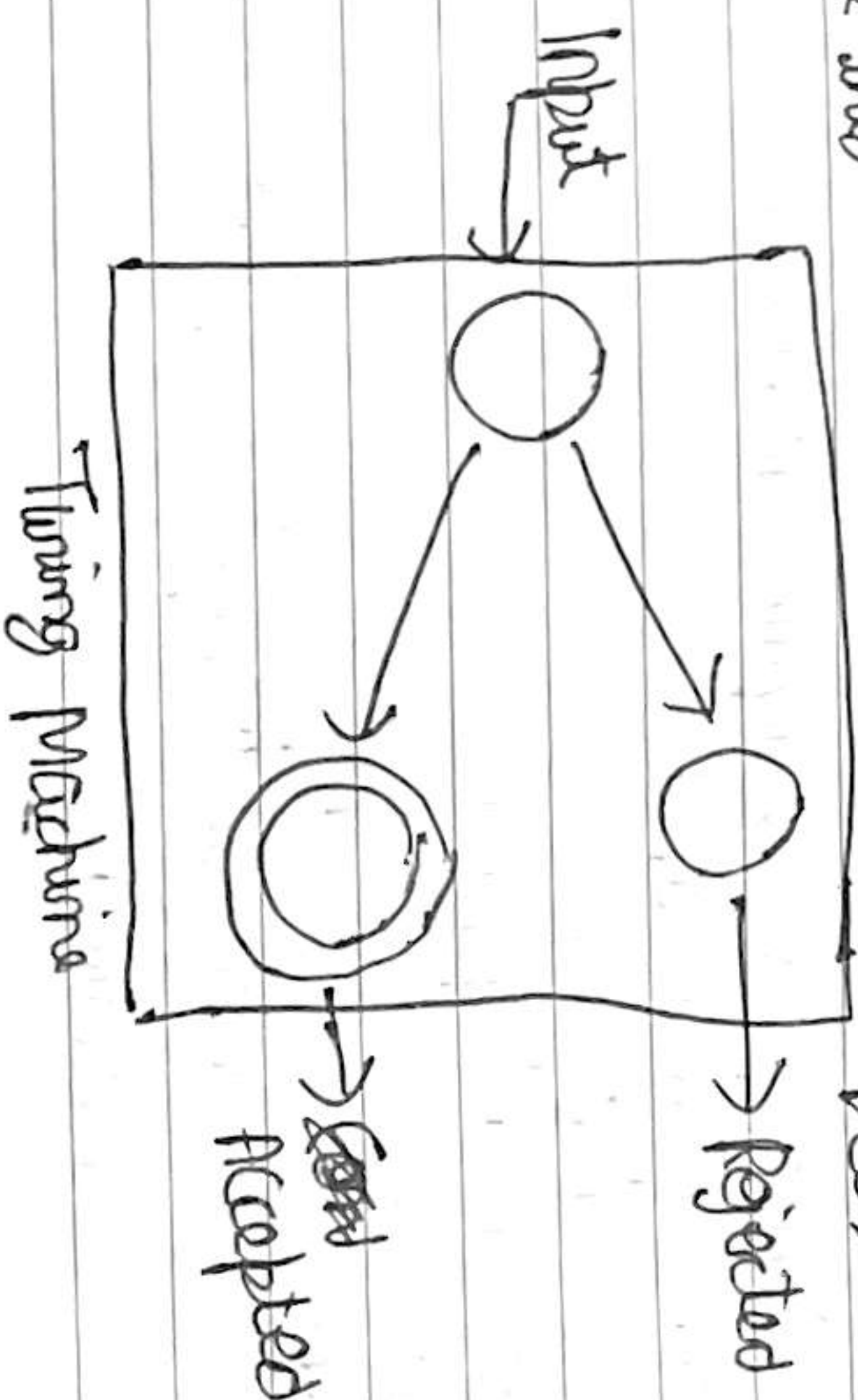
- 1) A language is called Decidable (or recursive) if there is a Turing machine which accepts & halts on every input string w. Every decidable language is Turing Acceptable



- 2) A decision problem P is decidable if the language L of all yes instances to P is decidable

→ there exists a TM that always halts & correctly decides whether an input belongs to L or not

- 3) For a decidable language, for each input string, the TM halts either at the accept or reject state like this -



Q. Find out whether the following problem is decidable or not.

Is a number 'm' prime?

Ans Prime = {2, 3, 5, 7, 11, 13, ...}

Number

Divide the number 'm' by all the numbers between 12 and 'm' starting from 2

If a number divides m completely (remainder = 0), then m is not prime

If any divisor is found $\Rightarrow$ Rejected state

else $\Rightarrow$ Accepted state

Since, this process always halts (event after checking up to 'm') and gives a definite answer $\Rightarrow$ It is a decidable problem

36
18 2
12 3
9 4
6 6

Since, this process always halts (event after checking up to 'm') and gives a definite answer $\Rightarrow$ It is a decidable problem

Undecidable language -

1) No Turing machine can decide every input string for these languages

2) A problem is undecidable if its language is not decidable

3) Undecidable languages are not recursive, but may be recursively enumerable

4) TM may halt for some inputs but not all

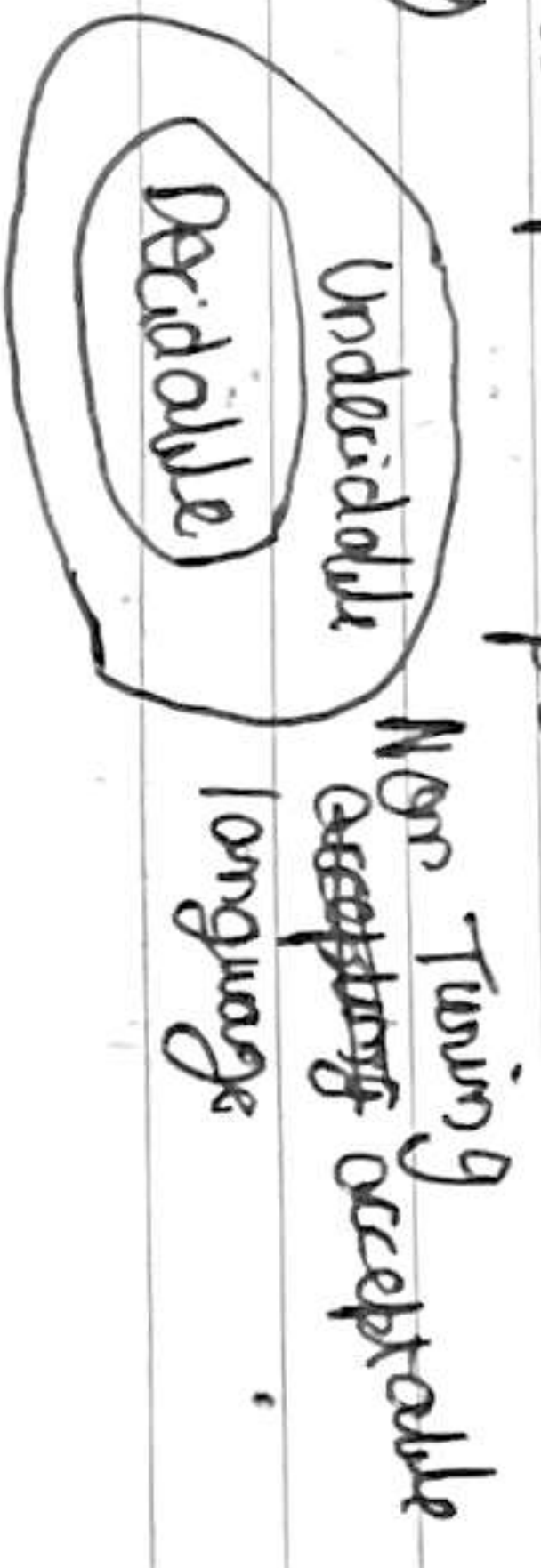
5) Eg- Halting problem of TM

Post Correspondence problem

Minimal Index problem

Post Correspondence problem

6) Hierarchy



Decidable Problems = TM halts for all inputs
Semi decidable = TM halts only for accepted inputs
Undecidable problems = TM doesn't halt or decide for all inputs

Decidable vs Undecidable problems -

Aspect	Decidable	Undecidable
Definition	Solved by an algo in finite time	No algorithm works for all cases
Solvability	Always solvable step by step	Not solvable for every input
Algorithm	Exists & always halts	May not exist or may not halt
Halting	Always stops for every input	Might never stop or loop forever
Eg	Checking prime even/odd	Halting problem, Post Correspondence problem
Decision procedure	Clear method to reach a conclusion	No guaranteed method for all cases
Complexity	May be complex but computable	Too complex or undecidable
Applications	Used in practical computing tasks	Shows limits of computing

Reducibility -

1) Allows us to compare the complexity or decidability of problems. Tells us whether solving one problem can help in solving another

2) A language (or problem) A is reducible to language B (written as $A \leq B$) if we can convert instances of A into instances of B such that:

(i) The answer to the instance of B gives the same answer (yes/no) for A

(ii) This is done using a computable function (for decidability) or polynomial time function (for complexity)

Types of Reducibility -

Type	Symbol	Used for	Function must be
Many one reducibility	$A \leq_m B$	Decidability, RE/Recursive sets	computable e.g.
Turing reducibility	$A \leq_T B$	More general than $\leq_m$	Oracle lower
Polynomial Time Reducibility	$A \leq_P B$	Complexity classes	Poly. time

Why reducibility is important?

- 1) ~~H-A~~ To prove Undecidability
- 2) To prove NP completeness
- 3) To transfer knowledge

Characterization of Reducibility - Key Points -

- 1) Solving $A \leq$ Solving B:
If A is reducible to B, solving A is not harder than solving B
- 2) Decidability through Reduction:
If $A \rightarrow B$ and B is decidable, then A is also decidable
- 3) Undecidability through Reduction:
If A is undecidable and $A \rightarrow B$, then B is also undecidable

Algorithm -

Finite, well defined sequence of computational step that transforms a given input into a desired output

- Characterisation of an algorithm -
- (i) Input: zero or more inputs
 - (ii) Output: at least one output
 - (iii) Finiteness
 - (iv) Definiteness
 - (v) Effectiveness

Introduction to Computational Complexity Theory -

- 1) Studied time & space complexity of algorithms
- 2) Focus on P vs NP - whether every verifiable problem is also solvable in polynomial time

- P = Solvable in polynomial time
 - NP = Verifiable in polynomial time
 - NP complete = Hardest NP Problems; all NP problems reduce to them

- 3) Used in cryptography & optimization
- 4) Other classes: PSPACE, EXPTIME
- 5) Defined lower limits of algorithmic efficiency

Major problems in Computational Complexity -

- 1) P versus NP problem
- 2) Complexity of specific problems
- 3) Circuit lower bounds
- 4) De-randomization
- 5) Quantum complexity theory

Complexity classes -

- 1) P class
- 2) NP class
- 3) NP hard class
- 4) NP complete class

P class -

- P = Polynomial Time Problems
- Solved by a deterministic Turing machine in polynomial time
- Decision Problems (Yes/No type)

Time complexity: $O(n^k)$, $k = \text{constant}$

Tractable can be solved efficiently in theory & practice
eg. eg. Shortest path in a graph (Dijkstra's algorithm)

NP Class -

- NP = Non deterministic Polynomial Time
- Problems solved by a non deterministic machine in polynomial time
- Hard to find, but easy to verify solutions
- Verification can be done by a deterministic machine in polynomial time

Co-NP Class -

- Complement of NP class
- If the answer is No, the proof can be verified in polynomial time
- If a problem $X \in \text{NP}$, then its complement $X' \in (\text{Co-NP})$
- Needs to verify only one answer (Yes or No) in polynomial time
- eg. Checking if a number is prime
- Integer factorization

$P \subseteq \text{NP}$ -

- Every P problem can also be verified in NP
- Hence, $P \subseteq \text{NP}$
- Main question: Is $P = \text{NP}$?
- If $P = \text{NP}$ $\Rightarrow$ All verifiable problems can be solved quickly
- If $P \neq \text{NP}$ $\Rightarrow$ Some problems are easy to verify but hard to solve

P vs NP -

Aspect	P	NP
1) Solving time	Polynomial time	May take exponential time
2) Verifying time	Polynomial time	Polynomial time
3) Machine type	Deterministic TM	Non deterministic TM
4) eg.	Shortest path, sorting, Primality	SAT subset sum, Hamiltonian path

NP Hard Class -

- Problems at least as hard as the hardest NP problems
- Every problem in NP can be reduced to an NP Hard in polynomial time
- Not all NP hard problems are in NP
- Difficult to check or verify solutions
- eg. Subset Sum Problem
- Turing Halting problem
- Acts as a benchmark for ~~fast~~ program hardness in computational theory

Real World Applications -

- 1) Approximate computing
- 2) Cryptography
- 3) Data Mining
- 4) Routing/Vehicle Routing

NP complete = NP + NP Hard
Not all NP Hard are NP complete

NP Hard Vs NP Complete -

Aspect	NP Hard	NP Complete
1) Verification	No need for polynomial time verification	Can be verified in polynomial time
2) Type of Problem	Not necessarily a decision problem	Always a decision problem
3) Relation	Not all NP Hard are NP complete	All NP Complete are NP Hard
4) eg.	Halting problem, vertex cover problem	Hamiltonian cycle, Circuit Satisfiability